

Reducing Input Size

Case Study: Clustering (Part 1)

Twofold role of clustering:

- 1 Example of a tough problem requiring input size reduction
- 2 Summarization technique that can be used for input size reduction

OUTLINE

① Definition of clustering

② Basic notions:

- Metric space.
- Distance function.
- Optimization problem and approximation algorithm.

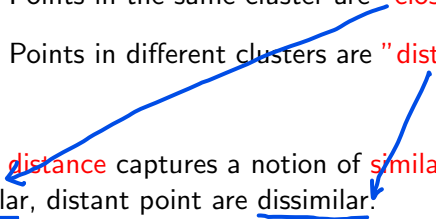
③ Center-based clustering

General definition

Given a **set of points** belonging to some **space**, with a notion of **distance** between points, **clustering** aims at grouping the points into a number of subsets (**clusters**) such that

- Points in the same cluster are "**close**" to one another
- Points in different clusters are "**distant**" from one another

The **distance** captures a notion of **similarity**: close points are similar, distant point are dissimilar.



A **clustering problem** is usually defined by requiring that clusters optimize a given **objective function**, and/or satisfy certain **properties**. Numerous clustering problems have been defined, studied and employed in applications over the years.

Example

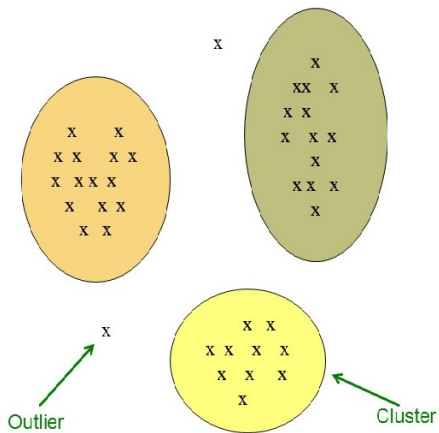


Figure from [Leskovec et al.: Mining Massive Datasets, 2014].

The clustering problem is a hard one!

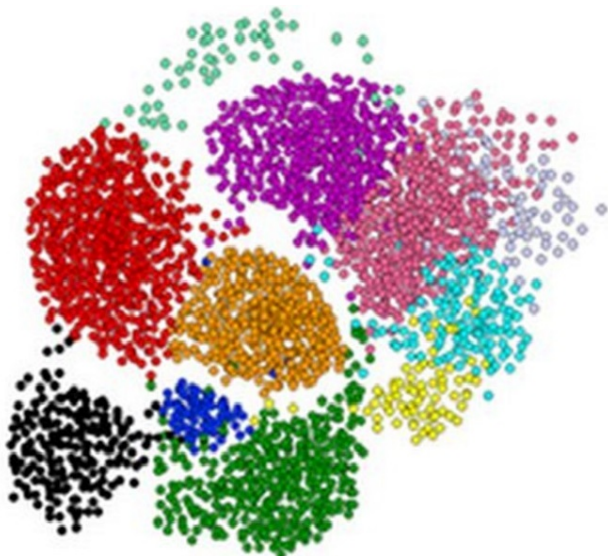


Figure from [Leskovec et al.: Mining Massive Datasets, 2014].

Applications

Clustering or cluster analysis is a fundamental task in data analysis, which finds applications in many different fields. Some examples:

- 1 Clustering is often a useful preprocessing step in other analysis tasks. E.g., it can be used for summarization, compression, outlier detection, class identification
- 2 Marketing. Clustering is used to partition consumers (potential customers) into segments based on shared characteristics. Then, high-yield segments can become targets of marketing campaigns or new products development.
- 3 Biology. Clustering of protein primary structures (amino acid sequences) is used to identify protein families and has important applications (e.g., in phylogenetic analysis)

Applications

- 1 **Image processing**. Clustering is used to analyze digital images for several purposes: e.g., object recognition; identification of different types of tissues in PET scans; identification of areas of similar land use in satellite pictures;
- 2 **Network analysis**. Clustering is used to detect communities
- 3 **Information retrieval**. Clustering is used to categorize documents or web pages based on their topics which are not explicitly given but are inferred from their contents
- 4 **Wireless sensor networks**. Clustering is used to identify suitable **cluster leaders** which can play the role of communication hubs. In general, this is an instance of the well know **facility location problem**.

Metric Space

Typically, the input of a clustering problem consists of a set of points from a **metric space**

Definition

A **metric space** is an ordered pair (M, d) where M is a set and $d(\cdot)$ is a **metric** on M , i.e., a function

$$d : M \times M \rightarrow \mathbb{R}$$

such that for every $x, y, z \in M$ the following holds

- $d(x, y) \geq 0$;
- $d(x, y) = 0$ if and only if $x = y$;
- $d(x, y) = d(y, x)$; (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$; (triangle inequality)



Remark: The choice of the function may have a significant impact on the effectiveness of the cluster analysis.

Distance functions

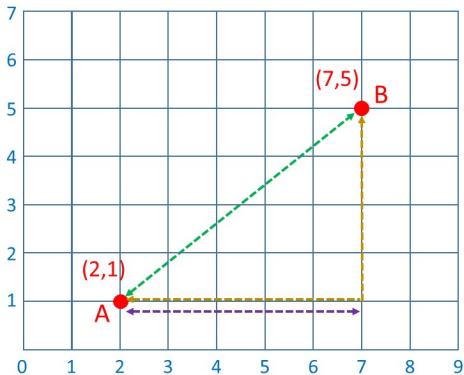
- **Minkowski distances.** Let $X, Y \in \mathbb{R}^n$, with $X = (x_1, x_2, \dots, x_n)$ and $Y = (y_1, y_2, \dots, y_n)$. For $r > 0$ the L_r – distance between X and Y (a.k.a. L_r – norm)

$$d_{L_r}(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^r \right)^{1/r}.$$

- $r = 2$ (*Euclidean distance*): is the standard distance in $\mathbb{R}^n$ (also denoted by $\|\cdot\|$)
- $r = 1$ (*Manhattan distance*): it is the sum of the absolute differences of coordinates in each dimension. Used in grid-like environments
- $r = \infty$ (*Chebyshev distance*): it is the maximum absolute differences of coordinates, over all dimensions (i.e., the limit for r that tends to ∞).

Distance functions

Examples of Minkowski distances: $p = 2$



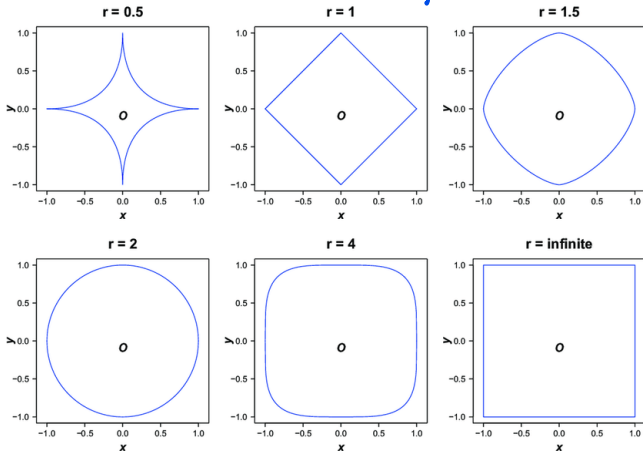
$$d_{l_1}(A,B) = (7-2) + (5-1) = 9$$

$$d_{l_2}(A,B) = (5^2 + 4^2)^{1/2} = 6.403..$$

$$d_{l_\infty}(A,B) = \max\{5,4\} = 5$$

Distance functions

Comparison of Minkowski distances: *points at distance ≤ 1 from center*



Picture from: Urquiza-Aguiar et al., Empirical Analysis of the Minkowski Distance Order in Geographical Routing Protocols for VANETs. Proc. WWIC, 2015.

Distance functions

- **Cosine (or angular) distance.** It is used when points are vectors in $\mathbb{R}^n$. Given $X = (x_1, x_2, \dots, x_n)$ and $Y = (y_1, y_2, \dots, y_n)$ in $\mathbb{R}^n$, their cosine distance is the angle between them, that is,

$$\begin{aligned}d_{\text{cosine}}(X, Y) &= \arccos \left(\frac{X \cdot Y}{\|X\| \cdot \|Y\|} \right) \\&= \arccos \left(\frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n (x_i)^2} \sqrt{\sum_{i=1}^n (y_i)^2}} \right)\end{aligned}$$

Example: For $X = (1, 2, -1)$ and $Y = (2, 1, 1)$ $n = 3$

$$d_{\text{cosine}}(X, Y) = \arccos \left(\frac{3}{\sqrt{6} \sqrt{6}} \right) = \arccos \left(\frac{1}{2} \right) = \frac{\pi}{3}$$

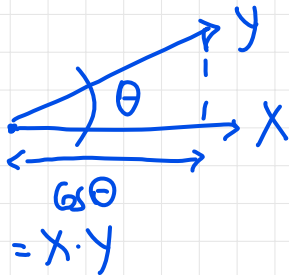
Remark. Often used in information retrieval to assess similarity between documents. A document X over a vocabulary of n words can be represented as an n -vector, where x_i is the number of occurrences of the i -th word of the vocabulary in X .

Distance functions

Intuition

Assume $\|X\| = \|Y\| = 1$

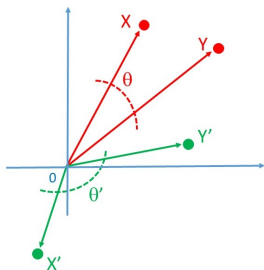
$$\Rightarrow d_{\text{cosine}}(X, Y) = \arccos(X \cdot Y)$$



Distance functions

Observations on the cosine distance

- For non-negative coordinates it takes values in $[0, \pi/2]$, while for arbitrary coordinates the range is $[0, \pi]$. Dividing by $\pi/2$ (or π) the range becomes $[0, 1]$.



$\theta \in [0, \pi/2]$ (coordinates ≥ 0)

$\theta' \in [0, \pi]$ (arbitrary coordinates)

- In order to satisfy the second property of distance functions for metric spaces, scalar multiples of a vector must be regarded as the same vector

Distance functions

- **Hamming distance**. Used when points are *binary vectors* over some n -dimensional space: $X, Y \in \{0, 1\}^n$. The Hamming distance between two vectors is **the number of coordinates in which they differ**.

$$d_{\text{Hamming}}(X, Y) = |\{i | x_i \neq y_i\}|$$

Example: for $X = (0, 1, 1, 0, 1)$ and $Y = (1, 1, 1, 0, 0)$

$$d_{\text{Hamming}}(X, Y) = 2 \quad (x_0 \neq y_0, \quad x_4 \neq y_4)$$

Observation: $d_{\text{Hamming}}(X, Y) = d_{L1}(X, Y)$

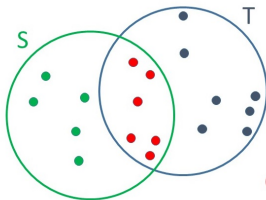
Distance functions

- Jaccard distance.** Used when points are sets (e.g., documents seen as bags of words). Let S and T be two sets over the same ground set of elements. The **Jaccard distance** between S and T is defined as

$$d_{\text{Jaccard}}(S, T) = 1 - \frac{|S \cap T|}{|S \cup T|}.$$

Note that the distance ranges in $[0, 1]$, and it is 0 iff $S = T$ and 1 iff S and T are disjoint. The value $|S \cap T|/|S \cup T|$ is referred to as the **Jaccard similarity** of the two sets.

Example:



$$d_{\text{Jaccard}}(S, T) = 1 - 6/18 = 2/3$$

Distance functions

Which distance should we use?

Minkowski

- ~~Euclidean~~ and Cosine distances are used when an object is characterized by the numerical values of its features (e.g., frequency).

Minkowski

- ~~Euclidean distance~~: it aggregates the gap in each dimension between two objects. Examples: GPS coordinates; measurements of physical/chemical processes.
- Cosine distance: it measures the ratios among features' values, rather than their absolute values. Example: relevance of a document since documents can have different lengths (objects $(100, 100, 100)$ and $(800, 800, 800)$ are equal, but they differ from $(100, 100, 50)$).

Distance functions

- **Hamming and Jaccard distances** are used when an object is characterized by having or not having some features (no multiplicity).
 - **Hamming distance**: it measures the total number of differences between two objects. Examples: fixed-length binary words, genetic distance.
 - **Jaccard distance**: it measures the ratio between the number of differences between two sets and the total number of points in the sets. Example: the topics describing a document.

Distance functions

Observation: All of the above distances satisfy the four requirements for a metric space. (Proving the triangle inequality for the Jaccard distance requires tedious yet simple calculations. If interested, see: *M. Levandowsky, D. Winter. Distance between sets. Nature 234, 1971.*)

Exercise

Show that the L_1 satisfies the four requirements for a metric space.

Curse of dimensionality

Consider a metric space with points in $\mathbb{R}^n$. The dimension n can be small (e.g., $n = 3$ for GPS data) but also very large (e.g., $n \gg 100$ for text documents).

The **curse of dimensionality** refers to issues that arise when processing data in high-dimensional spaces.

- **Quality:** Distance functions may lose effectiveness in assessing similarity;
- **Performance:** The running times may have a linear or even exponential dependency on n .

Curse of dimensionality

Quality

Random points in a high-dimensional metric space tend to be

- Sparse
- Almost equally distant from one another
- Almost orthogonal (as vectors) to one another.

As a consequence, in high dimensions distance functions may lose effectiveness in assessing similarity/dissimilarity. However, this holds in particular when points are random, and it might be less of a problem in some real-world datasets.

Curse of dimensionality

Performance.

- *Clustering*: In many cases, the running time of a clustering algorithm can be expressed as $O(C_{dist} \cdot N_{dist})$, where N_{dist} is the number of distance computations and C_{dist} is the cost of a distance computation ($C_{dist} = \Theta(n)$ for an n -dimensional Minkowski/Cosine/Hamming/Jaccard distances).
- *Nearest Neighbor Search problem*: given a set S of m points in $\mathbb{R}^n$, construct a data structure that, for a given query point $q \in \mathbb{R}^n$, efficiently returns the closest point $x \in S$ to q . The problem requires time:

$$\Theta(\min\{n \cdot m, 2^n\}).$$

Types of clusterings

Given a set of points in a metric space, a clustering problem often specifies an objective function to optimize.

The objective function also allows to compare different solutions.

Clustering problems

~~Objective functions~~ can be categorized based on whether or not

- A target number k of clusters is given in input.
- For each cluster a center must be identified.
- Disjoint clusters are sought.

Background: optimization problem

Optimization problem Π defined by

* Set of instances I ($\equiv$ inputs)

* $\forall i \in I$ set S_i of FEASIBLE SOLUTIONS

* Objective function Φ (optimal solution)

PROBLEM. Given $i \in I$ find $s \in S_i$ such that

$$\Phi(s) = \min \{ \Phi(s') \cdot s' \in S_i \}$$

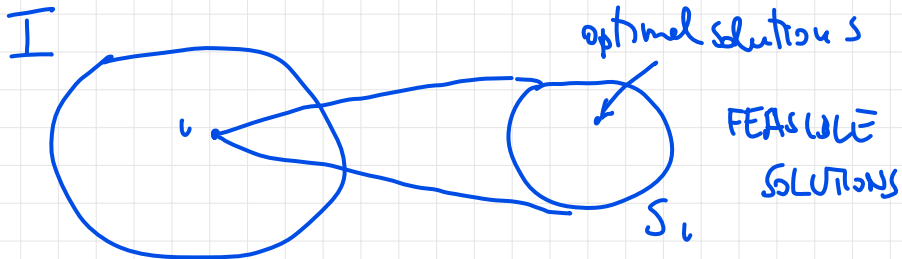
minimization problem

or

$$\Phi(s) = \max \{ \Phi(s') \cdot s' \in S_i \}$$

maximization problem

Background: optimization problem



s minimizes/maximizes the objective function Φ

Observation

For several **important optimization problems** no algorithms are known which find an optimal solution in time polynomial in the input size, and it is unlikely that such algorithms exist (***NP-hard problems***).

Therefore, especially when thinking of big data, one has to aim at approximations of optimal solutions.

Background: approximation algorithm

Optimization problem Π

A c -approximation algorithm A for Π ($c \gg 1$)

is an algorithm such that

* $\forall I \in \mathcal{I}$ returns a solution $A(I) \in S_I$
such that

$$\phi(A(I)) \leq c \min_{S' \in S_I} \phi(S') \quad \text{MINIMIZATION PROBLEM}$$

$$\phi(A(I)) \geq \frac{1}{c} \max_{S' \in S_I} \phi(S') \quad \text{MAXIMIZATION PROBLEM}$$

Background: approximation algorithm

$C \equiv$ approximation factor or ratio

$A(i) \equiv$ C -approximate solution for i
or a C -approximation for the
optimal solution for i

Observation

The approximation introduces a flexibility in selecting the solution to a given instance, which algorithms can exploit to run more efficiently.

Example: The maximum pairwise problem on an input set of n points requires $\Theta(n^2)$ operations for an exact solution, but only $\Theta(n)$ operations for a 2-approximate solution.

Center-based clustering

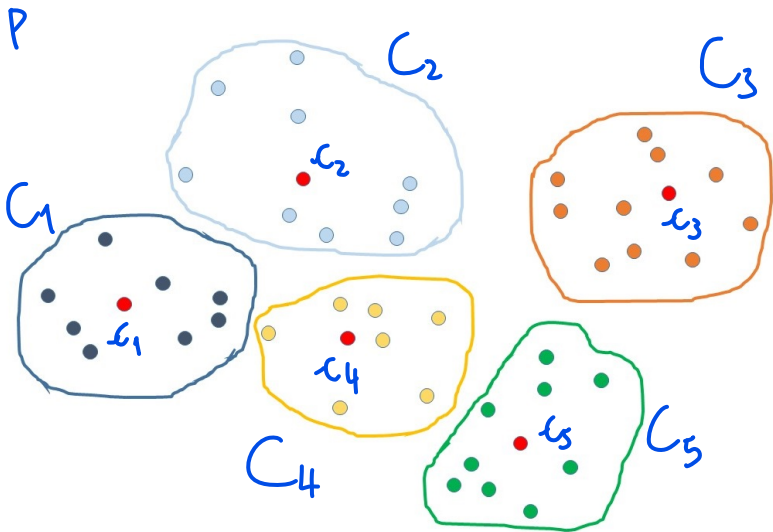
Center-based clustering

Let P be a set of N points in metric space (M, d) , and let k be the target number of clusters, $1 \leq k \leq N$. We define a k -clustering of P as a tuple $\mathcal{C} = (C_1, C_2, \dots, C_k; c_1, c_2, \dots, c_k)$ where

- $(C_1, C_2, \dots, C_k)$ defines a partition of P , i.e.,
 $P = C_1 \cup C_2 \cup \dots \cup C_k$
- $c_1, c_2, \dots, c_k$ are suitably selected centers for the clusters, where $c_i \in C_i$ for every $1 \leq i \leq k$.

Observe that the above definition requires the centers to belong to the clusters, hence to the pointset. Whenever appropriate we will discuss the case when the centers can be chosen more freely from the metric space.

Example of 5-clustering



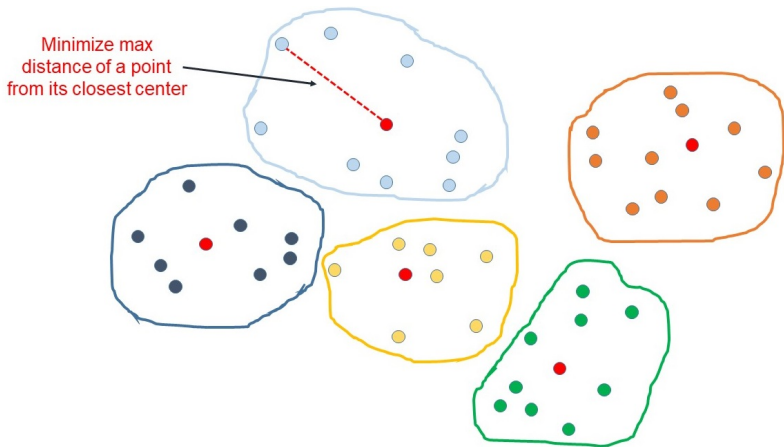
Center-based clustering

In this course, we study the 3 most popular center-based clustering problems (a formal definition will be given shortly):

- **k-center clustering:** requires to *minimize the maximum distance of any point from the closest center*
- **k-means clustering:** requires to *minimize the sum of the squared distances of the points from their closest centers*
- **k-median clustering:** requires to *minimize the sum of the distances of the points from their closest centers*

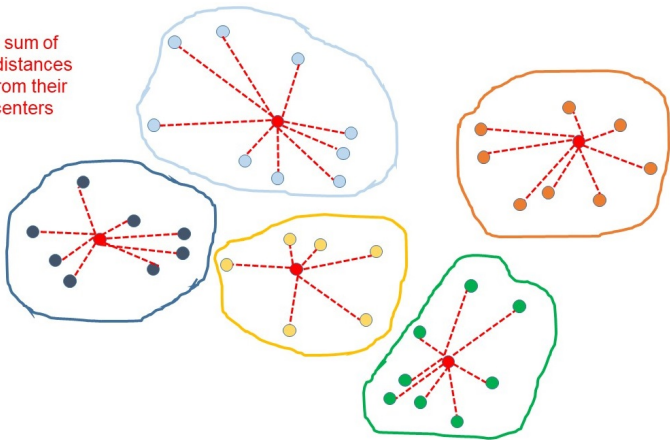
Observation: for k-means and k-median, minimizing the sum of (squared) distances is equivalent to minimizing the average (squared) distance.

Center-based clustering: k-center objective



Center-based clustering: k-means/median objectives

Minimize sum of
(squared) distances
of points from their
closest centers



Center-based clustering: formal definitions

Definition

Let $k > 0$. A k -clustering problem for a metric space (M, d) is an optimization problem that, given a finite pointset $P \subseteq M$, requires to determine a k -clustering $\mathcal{C}$ of P which minimizes a suitable objective function $\Phi(\mathcal{C})$.

The following are the three popular objective functions.

- $\Phi_{\text{kcenter}}(\mathcal{C}) = \max_{i=1}^k \max_{a \in C_i} d(a, c_i)$ (k-center clustering)
- $\Phi_{\text{kmeans}}(\mathcal{C}) = \sum_{i=1}^k \sum_{a \in C_i} (d(a, c_i))^2$ (k-means clustering).
- $\Phi_{\text{kmedian}}(\mathcal{C}) = \sum_{i=1}^k \sum_{a \in C_i} d(a, c_i)$ (k-median clustering).

k-means objective is also referred to as Sum of Squared Errors (SSE)

Center-based clustering: formal definitions

Input instance: pointset P and $k > 0$

Feasible solutions: all k -clusterings of P

Optimal solution: k -clustering $\mathcal{C}$ which
minimizes $\Phi_{k\text{-center}/k\text{-means}/\text{median}}(\mathcal{C})$

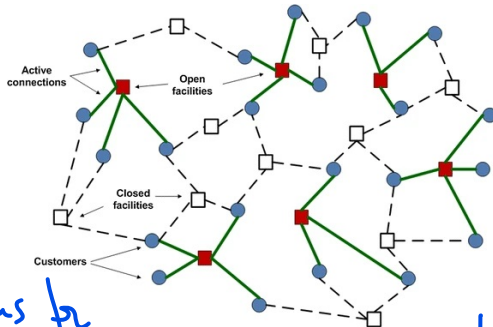
Obs several optimal clusterings may exist
for given P and k

Observations

- All aforementioned problems (k-center, k-means, k-median) are NP-hard $\Rightarrow$ it is impractical to search for optimal solutions.
- There are several efficient approximation algorithms that in practice return good-quality solutions. However, dealing efficiently with large inputs is still a challenge!
- How do we decide which problem suites better our needs?
 - k-center is useful when we need to guarantee that every point is close to a center (e.g., if centers represent locations of crucial facilities or if they are needed as "representatives" for the data). As we will see later, it is sensitive to noise.
 - k-means/median provide guarantees on the average (square) distances. k-means is more sensitive to noise but there exist faster algorithms to solve it.

Observations

- k-center and k-median belong to the family of **facility-location problems**



$k=6$

$\square \equiv$ locations for facilities
 $\bullet \equiv$ potential customers

$k =$ target number of facilities to be opened

Notations

For a pointset P from a metric space (M, d) , define:

- $\Phi_{\text{kcenter}}^{\text{opt}}(P, k) = \min\{\Phi_{\text{kcenter}}(\mathcal{C}) : \text{all } k\text{-clusterings } \mathcal{C} \text{ of } P\}.$
- $\Phi_{\text{kmeans}}^{\text{opt}}(P, k) = \min\{\Phi_{\text{kmeans}}(\mathcal{C}) : \text{all } k\text{-clusterings } \mathcal{C} \text{ of } P\}.$
- $\Phi_{\text{kmedian}}^{\text{opt}}(P, k) = \min\{\Phi_{\text{kmedian}}(\mathcal{C}) : \text{all } k\text{-clusterings } \mathcal{C} \text{ of } P\}.$

Also, for any point $p \in P$ and any subset $S \subseteq P$ we define

$$d(p, S) = \min\{q \in S : d(p, q)\}$$

Assignment primitive

Let P be a pointset and $S \subseteq P$ a set of k selected centers.

What is the best k -clustering around the centers of S w.r.t. the k -center, k -means, and k -median objectives?

The best k -clustering around S is the one where each point is assigned to the cluster of the closest center, for all 3 objectives

Assignment primitive

Assign (P, S) Pointset P set $S \subseteq P$ of k centers

Let $S = \{c_1, c_2, \dots, c_k\}$

for $i \leftarrow 1$ to k do $C_i = \{c_i\}$

for each $x \in P - S$ do {

$l \leftarrow$ index of the center closest to x

break ties
arbitrarily

assign x to C_l

return $\mathcal{C} = (C_1, C_2, \dots, C_k, c_1, c_2, \dots, c_k)$

Obs $\mathcal{C}$ can be simply represented by storing
with each $x \in P$ its closest center

Exercise

Consider a pointset P of N points represented as a key-value pairs (ID_x, x) , where ID_x is a distinct integer in $[0, N)$, and x is a point. Let also S be a set of k centers. Assuming $k \leq \sqrt{N}$ and considering S as global data, show that $\text{Assign}(P, S)$ can be implemented in MapReduce in 1 round using $M_L = O(\sqrt{N})$ and $M_A = O(N)$. Use the suggested representation for $\mathcal{C}$

Map Phase · $(ID_x, x) \rightarrow (ID_x, (x, c(x)))$
 $c(x)$ = center of S closest to x , which can be computed by checking all distances between x and centers of S . $M_L = O(k) = O(\sqrt{N})$ for storage of S